iMovies - MERN Stack Web Application

Full Stack Development Project Documentation

Author / Developer: Harshita Kachhwahe

1. Introduction

Project Title: iMovies - Full Stack Movie Discovery & Booking Platform

Team Members:

- **Harshita Kachhwahe** - Full Stack Developer (Frontend UI/UX, Backend API Architecture, Database Schema, Authentication & Seat Reservation Logic).

2. Overview

Purpose: iMovies is a comprehensive full-stack MERN web application built to offer users a modern and fluid platform for discovering movies, checking showtimes, and booking cinema tickets. It brings together content filtering, user state management, and real-time booking tracking in an intuitive interface.

Features:

- **Dynamic Movie Catalog:** Browse movies filterable by genre, language, and search titles.
- **User Authentication:** Secure user sign-up and login using JWT (JSON Web Tokens) and encrypted passwords.
- **Showtime & Seat Selection:** Choose showtimes and reserve seats per movie.
- **Personalized "My Bookings" Section:** Track active ticket reservations, reference codes, seat numbers, and booking status (Confirmed / Cancelled).
- **Modern Dark Aesthetic:** High-contrast cinema UI designed with gradient accents.

3. Architecture

The application follows a decoupled client-server architecture built on the MERN stack:

- **Frontend:** Built with **React.js** using modular functional components, hooks, Context API for authentication state, and custom responsive styling.
- **Backend:** Built with **Node.js** and **Express.js**. Exposes RESTful API endpoints for user auth, movie feeds, and ticket bookings.
- **Database:** Powered by **MongoDB** with **Mongoose ORM** to define schemas and manage relationships between Users, Movies, and Bookings.

4. Setup Instructions

Prerequisites: Node.js installed, npm, and a MongoDB database connection string (MongoDB Atlas or Local MongoDB).

Installation Steps:

1. Clone the project repository:

```
git clone https://github.com/harshitakachhwahe371-sudo/IMovies-MERN-Project.git
cd IMovies-MERN-Project
```

2. Install Backend dependencies:

```
cd server
npm install
```

3. Install Frontend dependencies:

```
cd ../client
npm install
```

4. Configure environment variables (create a `.env` file inside the `server/` folder):

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

5. Folder Structure

```
IMovies-MERN-Project/
├── client/                  # React Frontend App
│   ├── public/              # Public assets & HTML template
│   └── src/
│       ├── components/      # UI elements (Navbar, MovieCards, BookingCard)
│       ├── pages/           # Main views (Home, Discover, Login, Bookings)
│       ├── context/         # Auth Context State
│       ├── App.js           # App Routing
│       └── index.js         # React Entry
└── server/                  # Node.js Express Backend
    ├── config/              # Database Connection (db.js)
    ├── controllers/         # API Route Logic (authController, movieController)
    ├── models/              # Schemas (User.js, Movie.js, Booking.js)
    ├── routes/              # Express Routes (authRoutes, movieRoutes, bookingRoutes)
    ├── middleware/          # Auth Protection Middleware
    └── server.js            # Main Express Server File
```

6. Running the Application

Run both frontend and backend development servers concurrently:

Frontend:

```
cd client
npm start
```

Runs on `http://localhost:3000`

Backend:

```
cd server
npm start
```

Runs on `http://localhost:5000`

7. API Documentation

Method	Endpoint	Description	Access
POST	/api/auth/register	Register new user account	Public
POST	/api/auth/login	Authenticate user & issue JWT token	Public
GET	/api/movies	Get list of all movies & showtimes	Public
POST	/api/bookings	Create ticket booking	Private (Auth Token Required)
GET	/api/bookings/my-bookings	Fetch user's booking history	Private (Auth Token Required)

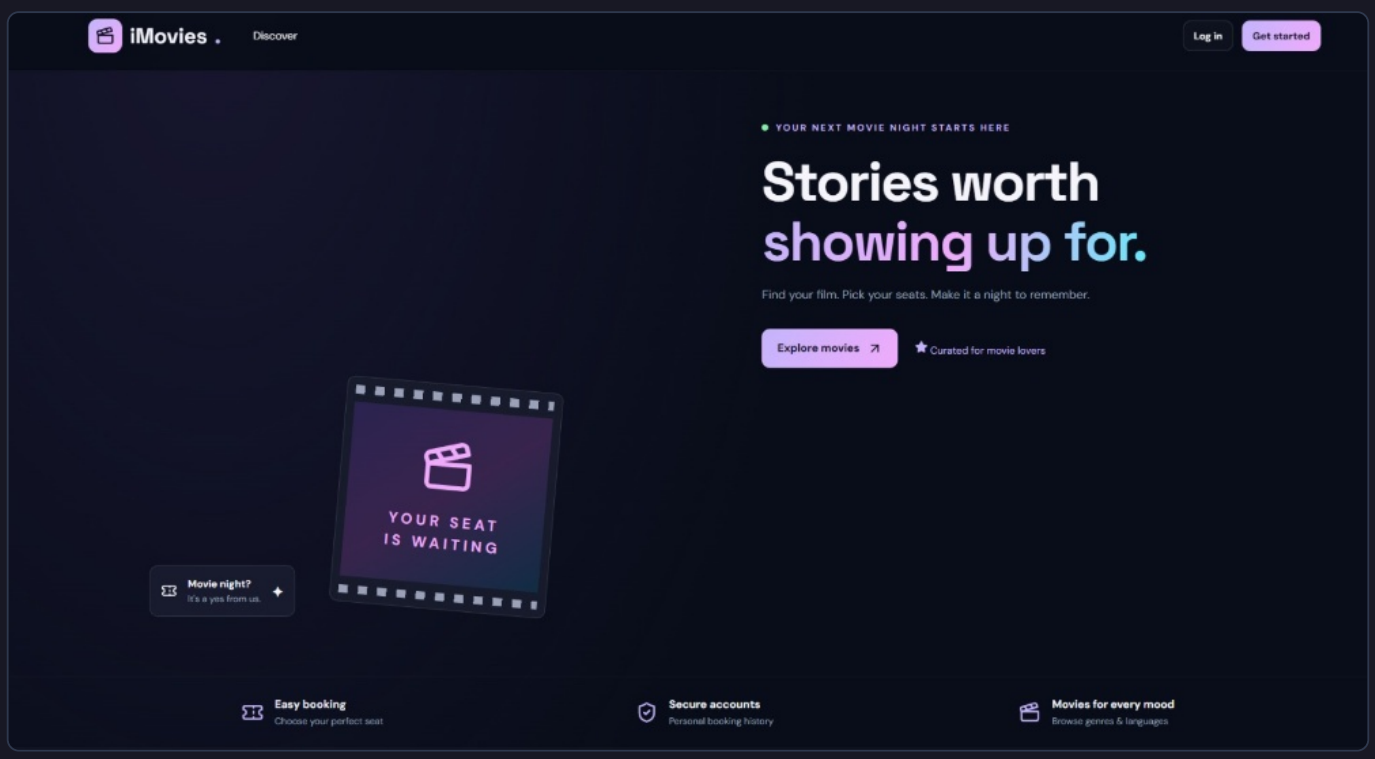
8. Authentication

The application implements JWT-based authentication:

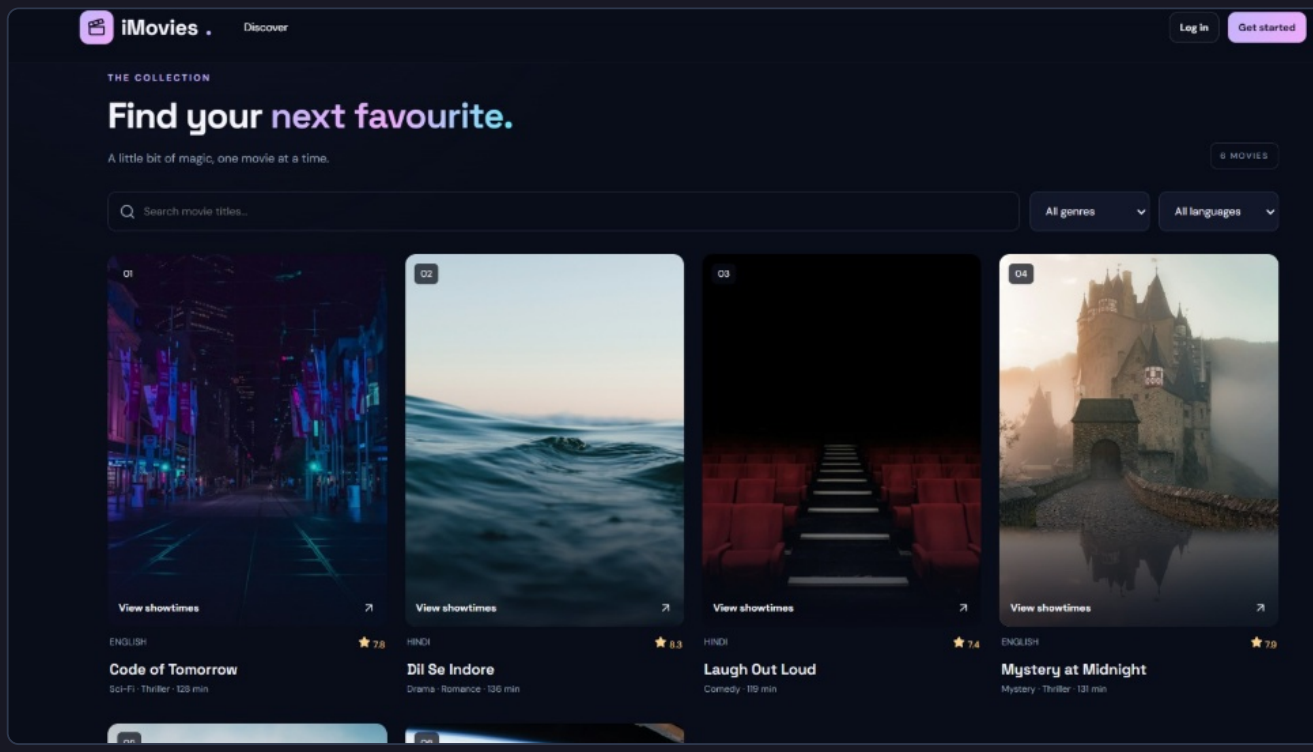
- Passwords are hashed using **bcryptjs** before being saved to MongoDB.
- Upon logging in, the server generates a signed **JSON Web Token (JWT)**.
- The frontend stores the token in `localStorage` and includes it in HTTP headers (`Authorization: Bearer `) for protected routes.

9. User Interface

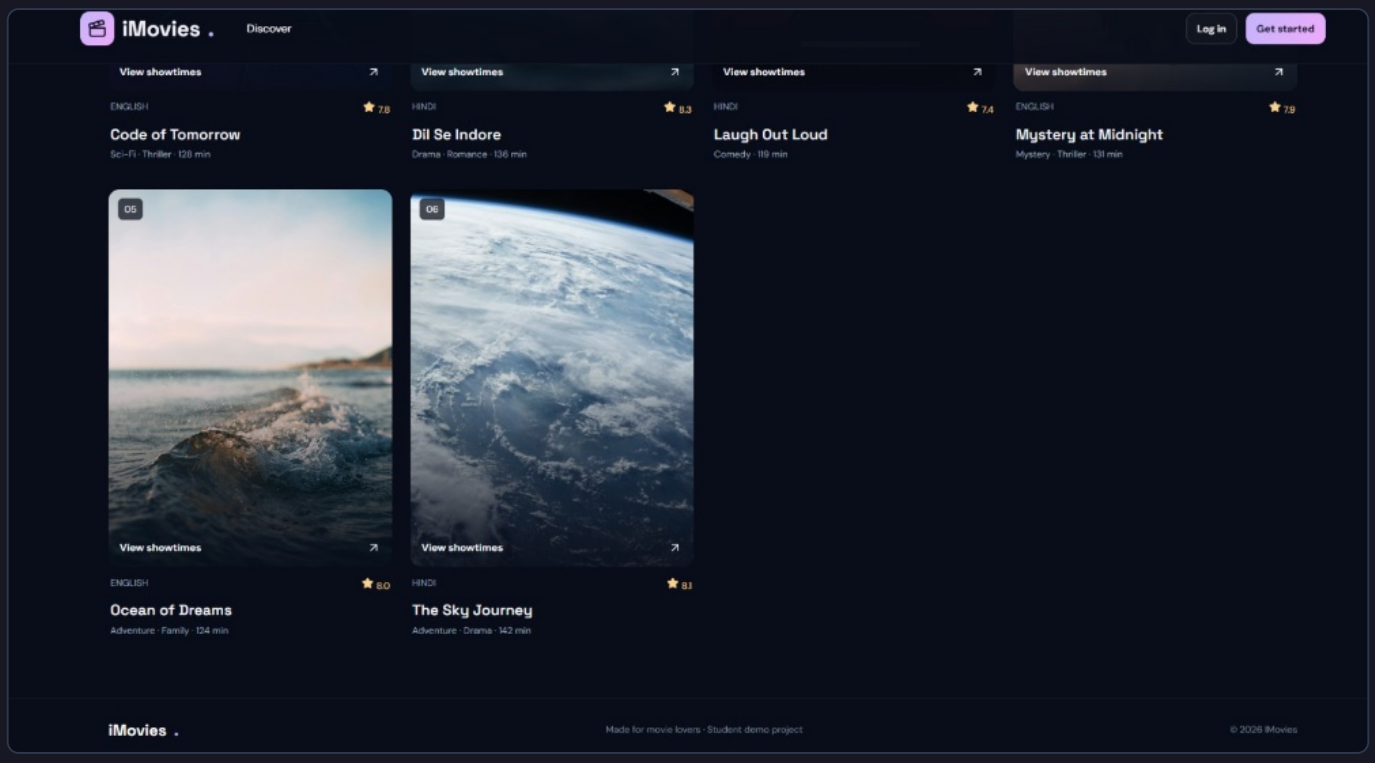
Below are screenshots showcasing the main UI features of the application.



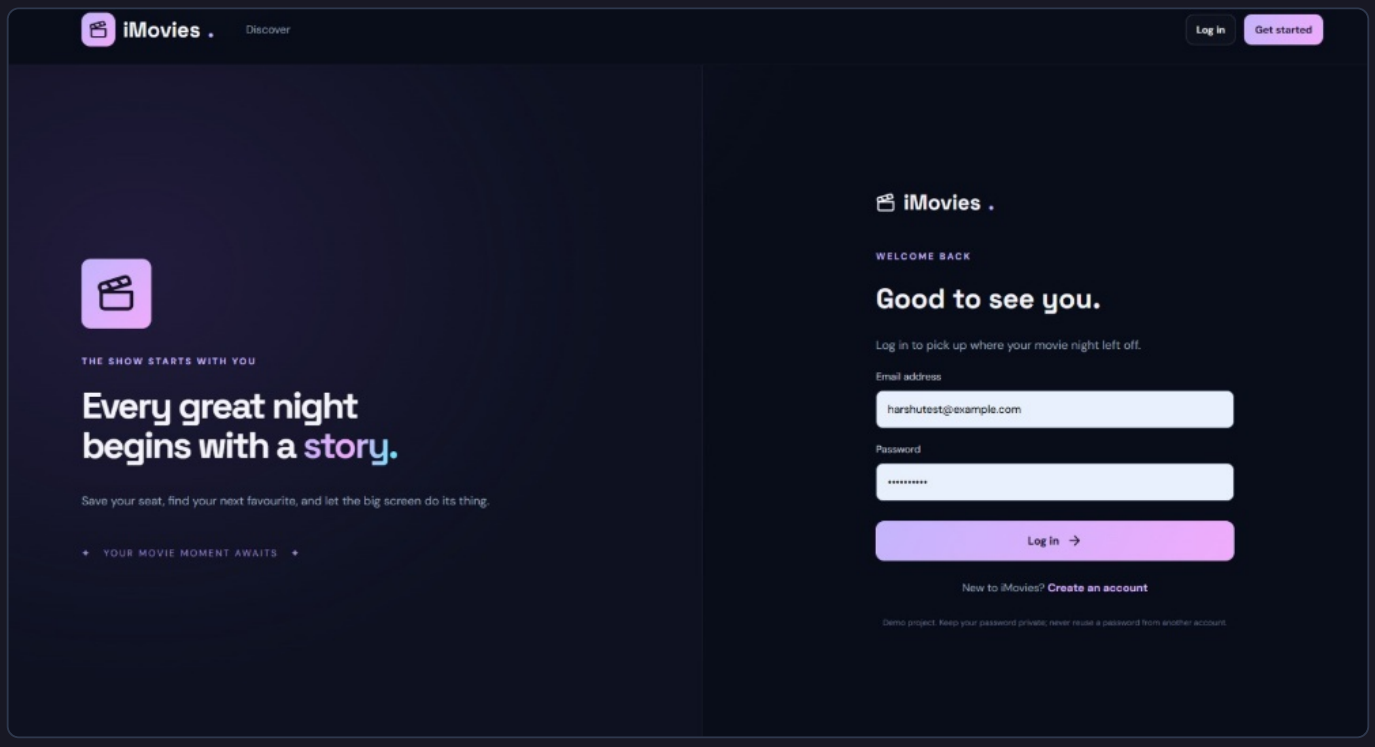
UI Feature 1: Landing / Hero Page - High-impact title banner, call-to-action buttons, and quick feature highlights.



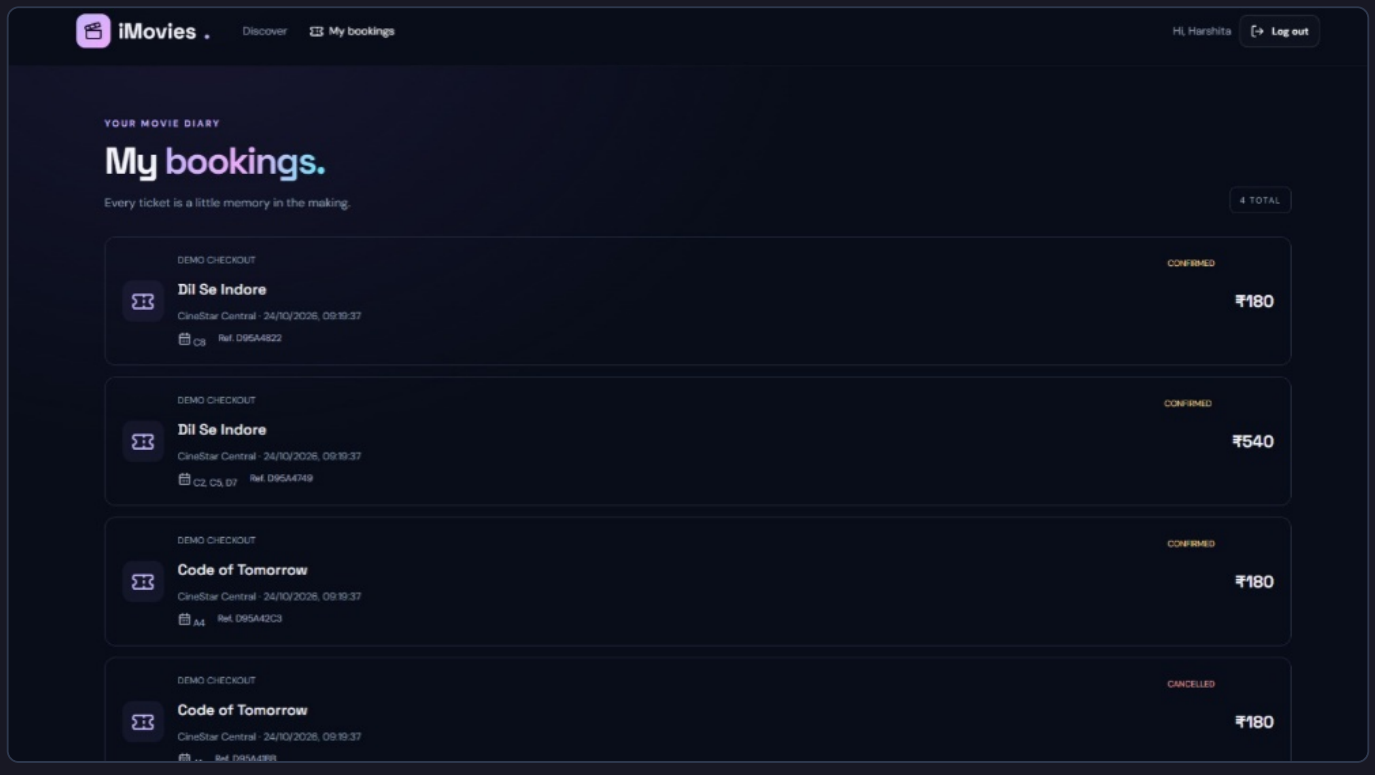
UI Feature 2: Movie Collection Grid - Search bar, Genre & Language filters, rating badges, and "View showtimes" buttons.



UI Feature 3: Extended Film Catalog showcasing movies with duration, ratings, and genre tags.



UI Feature 4: Authentication Page - Clean split-screen design for User Login and Account Creation.



UI Feature 5: My Bookings Diary - Real-time booking history display with reference codes, seat IDs, total pricing, and confirmation status.

10. Testing

- **API Endpoint Testing:** Validated all authentication, movie fetching, and booking routes using Postman.
- **Component & State Testing:** Ensured state updates properly on login, seat selection, and booking submission.

- **Cross-Browser Compatibility:** Verified layout consistency across Google Chrome, Microsoft Edge, and mobile viewports.

11. Screenshots or Demo

All core screenshots are integrated in Section 9 above. The code repository is hosted on GitHub for full review.

12. Known Issues

- Payment processing is currently configured in demo checkout mode.
- Seat layout selection uses standard default seat grid arrays in the current build.

13. Future Enhancements

- Integration with real payment gateways like Razorpay or Stripe.
- Interactive 3D seat mapping for cinema screens.
- Automated booking confirmation emails using Nodemailer.
- Admin Dashboard for managing movies, theaters, and ticket prices.